

# Exercise 13: Bayesian Optimization

## Optimization in Machine Learning

### Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

#### Python

```
pip install numpy==2.0.2 matplotlib==3.10.8 scipy==1.15.3 scikit-learn==1.7.2
```

#### R

```
install.packages("remotes")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("DiceKriging", "1.6.1")
remotes::install_version("patchwork", "1.3.0")
```

### Problem 1: Closed-form Expected Improvement

Derive the closed form expression of the **Expected Improvement** acquisition function:

$$a_{\text{EI}}(\mathbf{x}) = (f_{\min} - \hat{f}(\mathbf{x})) \Phi\left(\frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}\right) + \hat{s}(\mathbf{x}) \phi\left(\frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}\right).$$

Assume  $Y(\mathbf{x}) \sim \mathcal{N}(\hat{f}(\mathbf{x}), \hat{s}^2(\mathbf{x}))$ .

*Hints:*

- Start from  $a_{\text{EI}}(\mathbf{x}) = \mathbb{E}_y[\max\{f_{\min} - y, 0\}] = \int \max\{f_{\min} - y, 0\} p(y) dy$ .
- Decompose the integral by whether  $y < f_{\min}$  or  $y \geq f_{\min}$ .

- Substitute  $u = (y - \hat{f}(\mathbf{x}))/\hat{s}(\mathbf{x})$ .
- Use  $\int_{-\infty}^z u \phi(u) du = -\phi(z)$ .

**Solution.**

We start with

$$a_{\text{EI}}(\mathbf{x}) = \mathbb{E}_y(\max\{f_{\min} - y, 0\}) = \int_{-\infty}^{\infty} \max\{f_{\min} - y, 0\} p(y) dy.$$

Observe that

$$\max\{f_{\min} - y, 0\} = \begin{cases} f_{\min} - y, & \text{if } y < f_{\min}, \\ 0, & \text{otherwise.} \end{cases}$$

All contributions for  $y \geq f_{\min}$  are zero. Therefore, we can additively decompose the integral and it simplifies to

$$a_{\text{EI}}(\mathbf{x}) = \int_{-\infty}^{f_{\min}} (f_{\min} - y) p(y) dy.$$

$$\begin{aligned} \alpha_{\text{EI}}(\mathbf{x}) &= \int_{-\infty}^{f_{\min}} (f_{\min} - y) p(y) dy \\ &= \int_{-\infty}^{f_{\min}} (f_{\min} - y) \frac{1}{\sqrt{2\pi\hat{s}(\mathbf{x})^2}} \exp\left(-\frac{(y - \hat{f}(\mathbf{x}))^2}{2\hat{s}(\mathbf{x})^2}\right) dy \\ &= \int_{-\infty}^z (f_{\min} - \hat{f}(\mathbf{x}) - u\hat{s}(\mathbf{x})) \frac{1}{\sqrt{2\pi\hat{s}(\mathbf{x})^2}} \exp\left(-\frac{u^2}{2}\right) \hat{s}(\mathbf{x}) du \quad \left(\text{Def. } u := \frac{y - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}, \frac{du}{dy} = \frac{1}{\hat{s}(\mathbf{x})}, z := \frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}\right) \\ &= \int_{-\infty}^z (f_{\min} - \hat{f}(\mathbf{x}) - u\hat{s}(\mathbf{x})) \phi(u) du \\ &= \int_{-\infty}^z (f_{\min} - \hat{f}(\mathbf{x})) \phi(u) du - \int_{-\infty}^z (u\hat{s}(\mathbf{x})) \phi(u) du \end{aligned}$$

Note that

$$\Phi(z) = \int_{-\infty}^z \phi(u) du$$

by definition.

Therefore, regarding the first integral:

$$\int_{-\infty}^z (f_{\min} - \hat{f}(\mathbf{x})) \phi(u) du = (f_{\min} - \hat{f}(\mathbf{x})) \Phi(z) = z \hat{s}(\mathbf{x}) \Phi(z).$$

Regarding the second integral we use the identity

$$\int_{-\infty}^z u \phi(u) du = -\phi(z).$$

Putting both together we obtain:

$$\begin{aligned} \alpha_{\text{EI}}(\mathbf{x}) &= z \hat{s}(\mathbf{x}) \Phi(z) - \hat{s}(\mathbf{x}) (-\phi(z)) \\ &= z \hat{s}(\mathbf{x}) \Phi(z) + \hat{s}(\mathbf{x}) \phi(z) \\ &= (f_{\min} - \hat{f}(\mathbf{x})) \Phi \left( \frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})} \right) + \hat{s}(\mathbf{x}) \phi \left( \frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})} \right). \end{aligned}$$

## Problem 2: Implement Bayesian Optimization

Implement BO with a Gaussian-process surrogate and Expected Improvement acquisition.  
Goal: minimize

$$f : [0, 1] \rightarrow \mathbb{R}, \quad x \mapsto 2x \sin(14x).$$

Start with 4 points sampled uniformly at random and terminate after 10 total evaluations.

1. Write the BO loop in pseudocode.
2. Implement it. Optimize the EI via a univariate method (Brent's / `optimize`).

### Solution.

Let  $\mathcal{D}$  be the initial design consisting of  $(x^{[1]}, y^{[1]}), \dots, (x^{[4]}, y^{[4]})$ . Set  $t$  to 4.

While  $t < 10$ :

1. Fit surrogate model on  $\mathcal{D}$ .
2. Optimize the Expected Improvement  $a_{\text{EI}}(x)$  to obtain a new point  $x^{[t+1]} := \arg \max_{x \in [0, 1]} a_{\text{EI}}(x)$ .
3. Evaluate  $x^{[t+1]}$  and update design data  $\mathcal{D} = \mathcal{D} \cup \{(x^{[t+1]}, f(x^{[t+1]}))\}$ .
4. Set  $t$  to  $t + 1$ .

Return  $x$  that minimizes  $f(x)$  in  $\mathcal{D}$ :  $\arg \min_{(x, y) \in \mathcal{D}} y$ .

### **i** Note

**R vs. Python.** Both use a Gaussian-kernel GP surrogate with the same closed-form EI, but they draw the 4 initial points from different RNGs (R's `runif` vs numpy's), so the specific evaluations - and hence the exact GP posterior and its uncertainty bands - differ between the panels. The method and where BO converges are the same.

## R

```
library(DiceKriging)
library(ggplot2)
library(patchwork)

set.seed(308)
f <- function(x) 2 * x * sin(14 * x) # objective

# Initial design: 4 random points in [0, 1]
initial_x <- runif(4)
design <- data.frame(x = initial_x, y = f(initial_x))
t <- 4

# Expected improvement using the closed form derived in problem 1.
ei <- function(x, current_fmin, current_gp) {
  pr <- predict(current_gp, newdata = data.frame(x = x), type = "SK")
  diff <- current_fmin - pr$mean
  z <- diff / pr$sd
  diff * pnorm(z) + pr$sd * dnorm(z)
}

while (t < 10) {
  gp <- km(design = design[, 1L, drop = FALSE], response = design[, 2L],
           covtype = "gauss", nugget = 1e-8, control = list(trace = FALSE))
  fmin <- min(design$y)
  x_new <- optimize(ei, interval = c(0, 1), maximum = TRUE,
                   current_fmin = fmin, current_gp = gp)$maximum
  design <- rbind(design, data.frame(x = x_new, y = f(x_new)))
  t <- t + 1
}

bo_best <- design[which.min(design$y), ]
```

```

cat(sprintf("BO best after 10 evals: x = %.4f, f = %+.4f\n",
           bo_best$x, bo_best$y))

# Objective with BO's evaluations (light -> dark = evaluation order)
x_grid <- seq(0, 1, length.out = 401)
df_curve <- data.frame(x = x_grid, y = f(x_grid))
df_design <- design
df_design$step <- seq_len(nrow(df_design))

ggplot() +
  geom_line(data = df_curve, aes(x, y), color = "#0033A0", linewidth = 0.5) +
  geom_point(data = df_design, aes(x, y, color = step), size = 2.4) +
  geom_point(data = bo_best, aes(x, y), color = "#D90012",
            size = 3.5, shape = 4, stroke = 1.5) +
  scale_color_gradient(low = "grey80", high = "#F2A800") +
  ylab("f(x)") +
  ggtitle("BO evaluations on the objective (light->dark = order, red X = best)")

# GP posterior at the final BO iteration (mean +/- 2*sd over the grid)
final_gp <- km(design = design[, 1L, drop = FALSE], response = design[, 2L],
              covtype = "gauss", nugget = 1e-8, control = list(trace = FALSE))
pred <- predict(final_gp, newdata = data.frame(x = x_grid), type = "SK")
df_post <- data.frame(x = x_grid, mean = pred$mean,
                    lo = pred$mean - 2 * pred$sd,
                    hi = pred$mean + 2 * pred$sd)

# Expected improvement induced by that surrogate;
# BO would query its argmax next.
ei_grid <- ei(x_grid, min(design$y), final_gp)
x_ei_next <- x_grid[which.max(ei_grid)]
df_ei <- data.frame(x = x_grid, ei = ei_grid)

p_post <- ggplot() +
  geom_ribbon(data = df_post, aes(x, ymin = lo, ymax = hi),
            fill = "#F2A800", alpha = 0.35) +
  geom_line(data = df_post, aes(x, mean), color = "#F2A800", linewidth = 0.7) +
  geom_line(data = df_curve, aes(x, y), color = "#0033A0", linewidth = 0.5) +
  geom_point(data = design, aes(x, y), color = "black", size = 1.8) +
  geom_vline(xintercept = x_ei_next, linetype = "dashed",
            color = "#D90012", linewidth = 0.5) +
  ylab("f(x)") +
  ggtitle("GP posterior + EI acquisition (dashed = argmax EI)") +

```

```

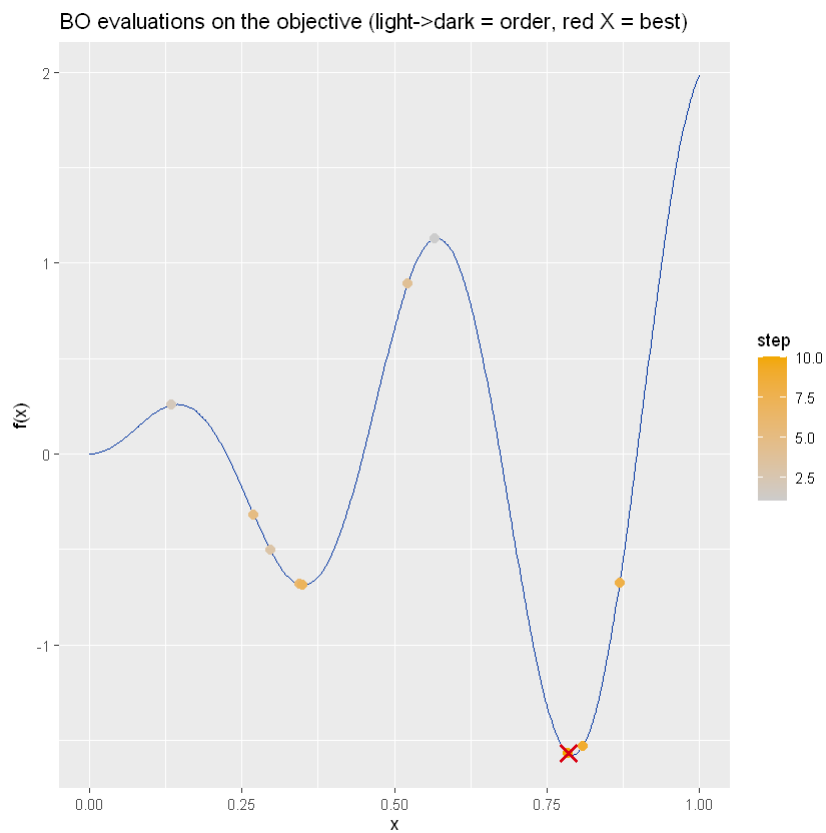
theme(axis.title.x = element_blank(), axis.text.x = element_blank())

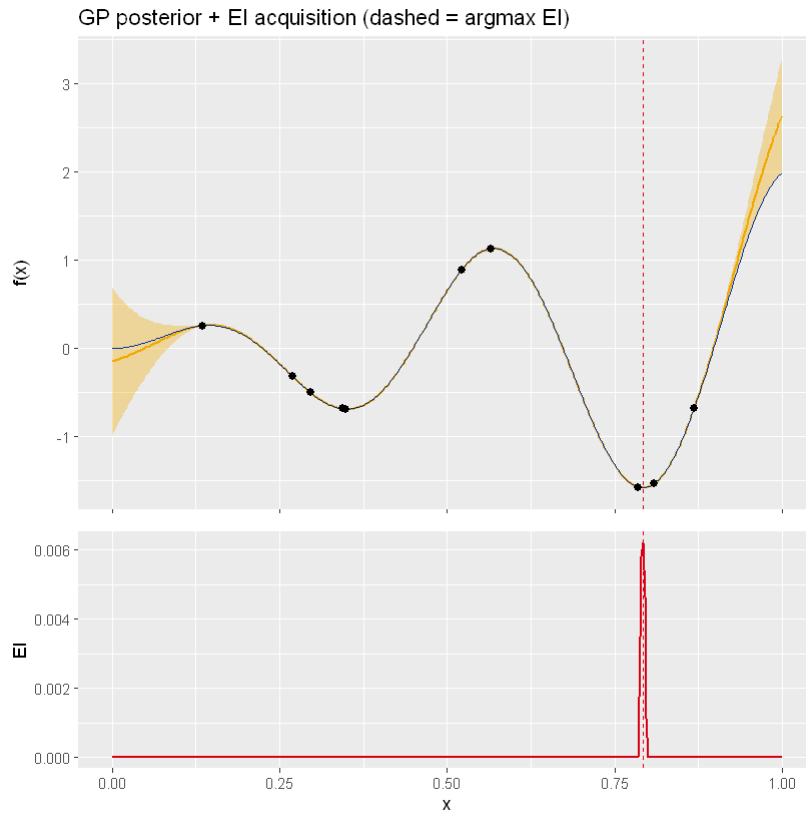
p_ei <- ggplot(df_ei, aes(x, ei)) +
  geom_line(color = "#D90012", linewidth = 0.6) +
  geom_vline(xintercept = x_ei_next, linetype = "dashed",
            color = "#D90012", linewidth = 0.5) +
  ylab("EI") + xlab("x")

print((p_post / p_ei) + plot_layout(heights = c(2, 1)))

```

BO best after 10 evals:  $x = 0.7854$ ,  $f = -1.5708$





## Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import minimize_scalar
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import ConstantKernel, RBF

def f(x):
    """Objective: 2x sin(14x) on [0, 1]."""
    return 2 * x * np.sin(14 * x)

# Gaussian-kernel GP surrogate, matching R's DiceKriging covtype = "gauss".
```

```

kernel = ConstantKernel(1.0) * RBF(length_scale=0.1)

def fit_gp(xd, yd):
    gp = GaussianProcessRegressor(kernel=kernel, alpha=1e-8, normalize_y=True)
    gp.fit(xd[:, None], yd)
    return gp

# Expected improvement using the closed form derived in problem 1.
def ei(x, current_fmin, gp):
    mu, sd = gp.predict(np.array([[x]]), return_std=True)
    mu, sd = mu[0], max(sd[0], 1e-9)
    diff = current_fmin - mu
    z = diff / sd
    return diff * norm.cdf(z) + sd * norm.pdf(z)

# Initial design: 4 random points in [0, 1]
np.random.seed(308)
x_design = np.random.uniform(size=4)
y_design = f(x_design)
t = 4

while t < 10:
    gp = fit_gp(x_design, y_design)
    fmin = y_design.min()
    res = minimize_scalar(lambda x: -ei(x, fmin, gp),
                          bounds=(0, 1), method="bounded")
    x_design = np.append(x_design, res.x)
    y_design = np.append(y_design, f(res.x))
    t += 1

best = int(np.argmaxmin(y_design))
print(f"BO best after 10 evals: x = {x_design[best]:.4f}, "
      f"f = {y_design[best]:+.4f}")

# Objective with BO's evaluations (light -> dark = evaluation order)
x_grid = np.linspace(0, 1, 401)
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(x_grid, f(x_grid), color="#0033A0", linewidth=0.6)
sc = ax.scatter(x_design, y_design, c=np.arange(len(x_design)), cmap="YlOrBr",

```



```

        s=45, edgecolor="k", linewidth=0.4)
ax.scatter(x_design[best], y_design[best], color="#D90012", marker="x",
           s=140, linewidths=2)
ax.set_xlabel("x"); ax.set_ylabel("f(x)")
fig.colorbar(sc, ax=ax, label="call index")
ax.set_title("B0 evaluations (light->dark = order, red X = best)")
plt.show()

# GP posterior at the final iteration (mean +/- 2*sd over the grid)
final_gp = fit_gp(x_design, y_design)
mu, sd = final_gp.predict(x_grid[:, None], return_std=True)

# EI induced by that surrogate; B0 would query its argmax next.
fmin = y_design.min()
sd_safe = np.maximum(sd, 1e-9)
z = (fmin - mu) / sd_safe
ei_grid = (fmin - mu) * norm.cdf(z) + sd_safe * norm.pdf(z)
x_ei_next = x_grid[int(np.argmax(ei_grid))]

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 6), sharex=True,
                              gridspec_kw={"height_ratios": [2, 1]})
ax1.fill_between(x_grid, mu - 2 * sd, mu + 2 * sd, color="#F2A800", alpha=0.35,
                 label="GP mean +/- 2 sd")
ax1.plot(x_grid, mu, color="#F2A800", linewidth=0.8, label="GP mean")
ax1.plot(x_grid, f(x_grid), color="#0033A0", linewidth=0.6, label="true f")
ax1.scatter(x_design, y_design, color="black", s=22, label="evaluations")
ax1.axvline(x_ei_next, linestyle="--", color="#D90012", linewidth=0.8)
ax1.set_ylabel("f(x)"); ax1.legend(loc="upper left", fontsize=8)
ax1.set_title("GP posterior + EI acquisition (Gaussian kernel)")
ax2.plot(x_grid, ei_grid, color="#D90012", linewidth=0.9)
ax2.axvline(x_ei_next, linestyle="--", color="#D90012", linewidth=0.8,
            label="argmax EI = next query")
ax2.set_xlabel("x"); ax2.set_ylabel(r"$a_{\mathrm{EI}}(x)$")
ax2.legend(loc="upper right", fontsize=8)
plt.show()

```

B0 best after 10 evals: x = 0.7898, f = -1.5766

